

Semantic Analysis

Team

Our team is **Tourists**

Team members: **Danila Khrankou & Tsimafei Kurstak**

Implementation

- **Language:** Projectl (Imperative)
- **Implementation:** C#
- **Module:** Handwritten semantic analyzer (`SemanticAnalyzer`) with symbol table, scopes, and a structural type system
- **Passes:** 2-pass analysis (Declarations $\rightarrow$ Type checking)

What the semantic analyzer does

After parsing the AST, the semantic analyzer validates:

- **Identifiers:** declarations before use, no duplicates in the same scope
- **Types:** resolution (primitive, user, array, record), compatibility, and inference
- **Routines:** parameters, return type, forward declarations, and that all paths return
- **Expressions/Statements:** type rules for arithmetic/logical/relational ops, assignments, control flow conditions
- **Arrays/Records:** index type and bounds checks (when statically known), field existence and types

Symbol table and scopes

- `SymbolTable` manages a stack of `Scope` s (global, routine bodies, loop variables).
- Each `Scope` maps names to `Symbol` s with `kind` (`Variable` , `Routine` , `Parameter`) and optional `Type` .
- Name resolution searches current scope up through parents; duplicates are rejected in the same scope.

```

_symbolTable.PushScope(routineDecl.Name);
// define parameters, locals, loop variables, etc.
_symbolTable.PopScope();

```

Type system (structural)

- Primitive: integer, real, boolean with relaxed compatibility:
 - integer is compatible with real (implicit widening)
 - integer is compatible with boolean (design decision to allow classic C-like truthiness if needed)
- Arrays: array[N] of T must match in size and element type (strict)
- Records: structural compatibility based on identical field sets with pairwise compatible field types

```

// Examples
new PrimitiveType(PrimitiveKind.Integer).IsCompatibleWith(new PrimitiveType(PrimitiveKi
new ArrayType(T, 10).IsCompatibleWith(new ArrayType(T, 10)); // true if T compatible
new RecordType({ x: integer, y: real }) ≡ new RecordType({ x: integer, y: real }); // s

```

Type resolution (AST TypeNode → semantic Type)

- PrimitiveTypeNode: look up built-in or user-defined type symbol
- UserTypeNode: resolve by name (detect unknown or circular references)
- ArrayTypeNode: resolve element type; array size must be a constant integer expression
- RecordTypeNode: resolve field types; reject duplicates, unknown types

```

Type? ResolveTypeNode(TypeNode t) => t switch
{
    PrimitiveTypeNode p => LookupType(p.TypeName),
    UserTypeNode u      => LookupType(u.TypeName),
    ArrayTypeNode a      => new ArrayType(Resolve(a.ElementType), ConstInt(a.Size)),
    RecordTypeNode r     => new RecordType(ResolveFields(r.Fields)),
};

```

Pass 1 — Declarations in detail

- Types
 - Reject duplicate names in the same scope

- Detect circular type definitions with a resolution stack
- Routines
 - Enter symbol with parameters (as AST nodes) and declared return type
 - Detect duplicates; allow one forward declaration per name
 - On encountering the full definition later: require signature match (param names/types and return type)
 - After Pass 1: any forward declarations without a body are reported

```
if (_forwardDeclarations.ContainsKey(name) && IsFullDecl(now))
    require SignaturesMatch(forward, now);
else if (IsDuplicateInScope(name)) error;
```

Pass 2 — Declarations and statements

- Variables
 - If explicit type: check initializer compatibility
 - If no type: infer from initializer and synthesize a matching `TypeNode` (via `TypeNodeFactory`)
 - Enter as variable symbol in current scope

```
if (varDecl.Type == null)
{
    var inferred = DeriveType(varDecl.InitialValue);
    varDecl.Type = TypeNodeFactory.CreateFromType(inferred, varDecl.InitialValue, _symbol)
}
```

- Routines
 - Push routine scope
 - Enter parameters as symbols with resolved types
 - Type-check body; then require all paths return a value if a return type is declared

```
if (routineDecl.ReturnType != null)
    CheckAllPathsReturn(routineDecl.Body, routineDecl.Name);
```

Statements and expressions

- Assignments: target must be modifiable (identifier, record/array access chain); value type must be compatible with target type
- If/While conditions: must be boolean-compatible (boolean or integer by design)

- For loop:
 - Range must be a special record-like type { start: integer, end: integer }
 - Loop variable introduced in a nested scope as integer
- Return: must be inside a routine; value presence/type must match routine declaration

Expressions (bottom-up):

- Literals → fixed primitive types
- Identifiers → resolved from symbol table; annotate with symbol and type
- Binary ops:
 - + - * / : numeric rules and common type promotion (integer → real)
 - = <> : compatibility check, boolean result
 - < > <= >= : numeric operands, boolean result
 - and or : boolean/integer-compatible operands, boolean result
- Unary ops:
 - - : numeric only
 - not : boolean/integer-compatible → boolean
- Arrays:
 - Index type must be integer-compatible; constant index is bounds-checked if array size known
 - Result type is element type
- Records:
 - Field must exist; result type is the field type
- Routine calls:
 - Routine must exist and be a routine; if still forward-only, report it
 - Arity and argument types must match parameters; result type is routine's declared return type (or void)

Troubleshooting highlights

- Grammar fixes from earlier iterations
 - Problem: grammar allowed variable declarations without initialization; record fields were modeled as a generic list of variable declarations, which complicated typing and structure.
 - Solution: updated grammar so variables must have an initializer when type is omitted; record fields now use a dedicated field node, simplifying record type construction and validation.
- Forward declarations consistency

- Problem: accepting a forward declaration and a later full definition with mismatched signatures leads to subtle bugs.
- Solution: on full declaration, we compare parameter count, names, resolved types, and return type; mismatch → error.
- All-paths-return for functions
 - Problem: false negatives for `if` constructs (then/else) and trailing loops.
 - Solution: `AllPathsReturn` recursively explores statement lists and requires returns on both branches; loops at the end don't satisfy the requirement.
- Array size and index verification
 - Problem: array sizes must be constant integers, and indexes should be checked when statically known.
 - Solution: constant-expression evaluator for simple integer expressions; emit out-of-bounds errors for constants and a warning-like message for non-constant indexes when size is known.
- Integer-Boolean compatibility (design decision)
 - Problem: real-world code sometimes uses integers as booleans in conditions; being too strict causes friction.
 - Solution: allow `int↔bool` compatibility in logical contexts while keeping other checks sound (e.g., numeric ops remain numeric-only).